

An important design decision to make is whether ILP shall be carried out before or after register allocation; scheduling and register allocation are strongly dependent on each other.

We have three approaches for register allocation and instruction scheduling, graphically depicted in Figure 1. We will look at each of them in turn.

Method I: Register allocation before instruction scheduling

When register allocation is done before instruction scheduling, it is called early register assignment. The procedure is depicted in Figure 2. This method aims at making optimal use of available registers by re-using registers whenever possible, and it allocates registers without considering ILP. This can cause anti-dependencies to be added, which arise due to the re-definition of registers that occur when multiple values are mapped to the same physical register. These added anti-dependencies can restrict ILP. For example, let's suppose we have the following code:

```
1.  MOVE 10, [a] ;
2.  MOVE 11, [b] ;
3.  ADD 12, 11, 10 ;
4.  ADD 13, 10, 10 ;
5.  ADD 14, 12, 10 ;
```



Note: The above instruction set is a sample output of the code generation phase, and contains virtual registers (10, 11, 12, etc); register allocation has not yet been performed. *[a]* and *[b]* imply memory locations. At the time of code generation, symbolic names 'a' and 'b' are used because it is not known which address registers will finally be allocated. The first operand is taken as the destination operand, and the remaining are source operands.

Now, we apply Method I to the code mentioned in the above example. First we allocate registers, and then proceed to instruction scheduling. This is done as shown in Figure 3.

In Figure 3, we have done early register assignment—register allocation before instruction scheduling. Assuming each instruction takes one instruction cycle, we have five resultant instruction cycles. Here, three physical registers are used—R0, R1 and R2.

Register re-use is done to make optimal use of available registers. However, this can generate either a false dependency or anti-dependency. We see here that when register assignment is done, an anti-dependency is generated between Instructions 3 and 4 due to the fact that the physical register 'R1' is reused. This anti-dependency prevents Instruction 4 from executing before Instruction 3 and thus, instruction scheduling is restricted. Before register allocation was done, we had an ILP possibility between Instructions 2 and 4, which can no longer be achieved. (Assume that the hypothetical processor under consideration allows combining of an operation and a transfer instruction in parallel, and that we may combine a maximum of two instructions in parallel.)

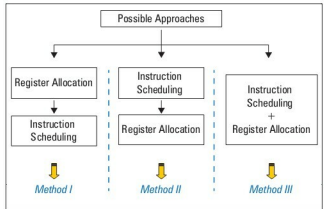


Figure 1: Possible approaches

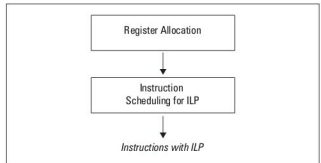


Figure 2: Register allocation before instruction scheduling

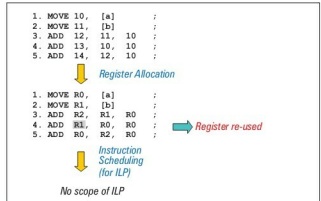


Figure 3: Method I used on code sample

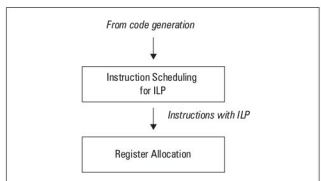


Figure 4: Register allocation after instruction scheduling